

LAPORAN PRAKTIKUM STRUKTUR DATA

DOUBLE LINKED LIST

Disusun Oleh:

Ihsanul Zaky EL-Muhammady

NIM:2511533001

Dosen Pengampu:

Dr.Wahyudi S.T , M.T

Asisten Laboratorium:

Zahran Azaria Anvaya



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG 2026

KATA PENGANTAR

Dengan penuh rasa syukur, saya mengucapkan terima kasih kepada Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum “Struktur Data Double Linked List (DLL)” ini dapat diselesaikan dengan baik.

Saya juga menyampaikan penghargaan kepada dosen pengampu Mata Kuliah Praktikum Struktur Data serta para asisten laboratorium yang telah memberikan arahan, bimbingan, dan bantuan selama proses praktikum berlangsung.

Laporan ini disusun sebagai salah satu bentuk dokumentasi kegiatan praktikum yang bertujuan untuk memahami konsep struktur data Double Linked List (DLL). Dalam laporan ini dibahas konsep dasar Double Linked List, prinsip kerjanya, serta implementasinya dalam bahasa pemrograman Java, mulai dari operasi penambahan node, penelusuran data, hingga penghapusan node, serta penerapannya dalam studi kasus sederhana seperti playlist musik.

Saya menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan agar laporan ini dapat menjadi lebih baik di masa yang akan datang, serta dapat memberikan manfaat dan menambah pemahaman mengenai struktur data Double Linked List.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	
DAFTAR ISI.....	
BAB I PENDAHULUAN	
1.1 Latar Belakang	
1.2 Tujuan	
1.3 Manfaat.....	
BAB II PEMBAHASAN	
2.1 NodeDLL_2511533001	
2.2 InsertDLL_2511533001	
2.3 PenelusuranDLL_2511533001	
2.4 HapusDLL_2511533001	
2.5 Tugas	
BAB III KESIMPULAN	
3.1 Kesimpulan	
3.2 Saran.....	
DAFTAR PUSTAKA.....	

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengelolaan data bukan hanya soal menyimpan nilai, tetapi juga bagaimana cara mengaksesnya secara efisien. Salah satu struktur data yang sering digunakan adalah Single Linked List, di mana setiap simpul (node) terhubung ke simpul berikutnya melalui sebuah pointer. Meskipun cukup berguna, Single Linked List punya keterbatasan yang cukup mengganggu, yaitu hanya bisa bergerak satu arah. Kalau butuh akses ke elemen sebelumnya, mau tidak mau penelusuran harus dimulai lagi dari awal.

Keterbatasan inilah yang kemudian memunculkan pengembangan struktur data yang lebih fleksibel, yaitu Double Linked List (DLL). Pada DLL, setiap simpul tidak hanya menyimpan pointer ke simpul berikutnya (next), tapi juga menyimpan pointer ke simpul sebelumnya (prev). Dengan begitu, penelusuran bisa dilakukan dari dua arah, baik dari depan ke belakang maupun sebaliknya, tanpa perlu mengulang dari awal.

Melalui praktikum ini, konsep Double Linked List dipelajari lebih dalam untuk memahami bagaimana struktur ini bekerja dan apa saja operasi yang bisa dilakukan di dalamnya, seperti insert, delete, dan traverse dua arah.

1.2 Tujuan

1. Memahami konsep dasar Double Linked List dan perbedaannya dengan Single Linked List.
2. Mengetahui cara kerja pointer *next* dan *prev* dalam menghubungkan antar simpul.
3. Memahami operasi dasar pada DLL seperti *insert*, *delete*, dan *traverse*.
4. Menganalisis keunggulan DLL dibandingkan struktur data linear lainnya dalam hal efisiensi akses data.
5. Mengembangkan logika pemrograman dalam menyelesaikan kasus yang membutuhkan traversal dua arah.

1.3 Manfaat

1. Meningkatkan pemahaman tentang cara kerja Double Linked List dalam pemrograman.
2. Melatih kemampuan mengelola pointer dan memori secara dinamis.
3. Membantu memahami penerapan DLL dalam kasus nyata seperti fitur *undo-redo*, navigasi browser, maupun playlist musik.
4. Memberikan gambaran bagaimana sistem yang butuh akses data dua arah bisa dirancang dengan lebih efisien.

BAB II

PEMBAHASAN

2.1 NodeDLL_2511533001

```
1 package pekan6_2511533001;
2
3 public class NodeDLL_2511533001 {
4     // mendefinisikan kelas node
5     int data_3001; // data
6     NodeDLL_2511533001 next_3001; // Pointer ke next node
7     NodeDLL_2511533001 prev_3001; // Pointer ke previous node
8
9     // konstruktor
10    public NodeDLL_2511533001 (int data_3001) {
11        this.data_3001= data_3001;
12        this.next_3001 = null;
13        this.prev_3001 = null;
14    }
15 }
16 }
17 }
```

Penjelasan:

Class ini berperan sebagai kerangka dasar node pada Double Linked List. Setiap node menyimpan tiga hal, yaitu data_3001 sebagai nilai yang disimpan, next_3001 sebagai penunjuk ke node berikutnya, dan prev_3001 sebagai penunjuk ke node sebelumnya. Karena ada dua pointer inilah DLL bisa ditelusuri maju maupun mundur, tidak seperti Single Linked List yang hanya bisa satu arah.

Pada bagian constructor, ketika node baru dibuat, nilai yang dimasukkan langsung tersimpan di data_3001. Sedangkan next_3001 dan prev_3001 awalnya diberi nilai null karena node tersebut belum terhubung ke node mana pun. Seiring node-node lain ditambahkan ke list, barulah kedua pointer ini akan diisi dengan referensi node yang sesuai.

2.2 InsertDLL_2511533001

```
1 package pekan6_2511533001;
2
3 public class InsertDLL_2511533001 {
4     // menambahkan node di awal DLL
5     static NodeDLL_2511533001 insertBegin_2511533001(NodeDLL_2511533001 head_3001, int data_3001) {
6         // buat node baru
7         NodeDLL_2511533001 new_node_3001 = new NodeDLL_2511533001(data_3001);
8         // jadikan pointer nextnya head
9         new_node_3001.next_3001 = head_3001;
10        // jadikan pointer prev head ke new_node
11        if (head_3001 != null) {
12            head_3001.prev_3001 = new_node_3001;
13        }
14        return new_node_3001;
15    }
16    // fungsi menambahkan node di akhir
17    public static NodeDLL_2511533001 insertEnd_2511533001(NodeDLL_2511533001 head_3001, int new_Data_3001) {
18        // buat node baru
19        NodeDLL_2511533001 newNode_3001 = new NodeDLL_2511533001(new_Data_3001);
20        // jika dll null jadikan head
21        if (head_3001 == null) {
22            head_3001 = newNode_3001;
23        }
24        else {
25            NodeDLL_2511533001 curr_3001 = head_3001;
26            while (curr_3001.next_3001 != null) {
27                curr_3001 = curr_3001.next_3001;
28            }
29            curr_3001.next_3001 = newNode_3001;
30            newNode_3001.prev_3001 = curr_3001;
31        }
32        return head_3001;
33    }
34    // fungsi menambahkan node diposisi tertentu
35    public static NodeDLL_2511533001 insertAtPosition_2511533001(NodeDLL_2511533001 head_3001, int pos_3001, int new_Data_3001)
```

```

36 // buat node baru
37 NodeDLL_2511533001 new_node_3001 = new NodeDLL_2511533001(new_Data_3001);
38 if (pos_3001 == 1) {
39     new_node_3001.next_3001 = head_3001;
40     if(head_3001 != null) {
41         head_3001.prev_3001 = new_node_3001;
42     }
43     head_3001 = new_node_3001;
44     return head_3001; }
45 NodeDLL_2511533001 curr_3001 = head_3001;
46 for (int i = 1; i < pos_3001 - 1 && curr_3001 != null; i++) {
47     curr_3001 = curr_3001.next_3001;
48 }
49 if (curr_3001 == null) {
50     System.out.println("Posisi tidak ada");
51     return head_3001; }
52 new_node_3001.prev_3001 = curr_3001;
53 new_node_3001.next_3001 = curr_3001.next_3001;
54 curr_3001.next_3001 = new_node_3001;
55 if (new_node_3001.next_3001 != null) {
56     new_node_3001.next_3001.prev_3001 = new_node_3001; }
57 return head_3001;
58 }
59 public static void printList_2511533001(NodeDLL_2511533001 head_3001) {
60     NodeDLL_2511533001 curr_3001 = head_3001;
61     while (curr_3001 != null) {
62         System.out.print(curr_3001.data_3001 + " <-> ");
63         curr_3001 = curr_3001.next_3001;
64     }
65     System.out.println();
66 }
67 public static void main(String[] args) {
68     // membuat dll 2 <-> 3 <-> 5
69     NodeDLL_2511533001 head_3001 = new NodeDLL_2511533001(2);
70     head_3001.next_3001 = new NodeDLL_2511533001(3);
71     head_3001.next_3001.prev_3001 = head_3001;
72
73     head_3001.next_3001.next_3001 = new NodeDLL_2511533001(5);
74     // cetak DLL di awal
75     System.out.print("DLL Awal: ");
76     printList_2511533001(head_3001);
77     // tambah 1 di awal
78     head_3001 = insertBegin_2511533001(head_3001, 1);
79     System.out.print (
80         "simpul 1 ditambah di akhir: ");
81     printList_2511533001(head_3001);
82     // tambah 6 di akhir
83     System.out.print (
84         "simpul 6 ditambah di akhir: ");
85     int data_3001 = 6;
86     head_3001 = insertEnd_2511533001(head_3001, data_3001);
87     printList_2511533001(head_3001);
88     // menambahkan node 4 di posisi 4
89     System.out.print("tambah node 4 di posisi 4: ");
90     int data2_3001 = 4;
91     int pos_3001 = 4;
92     head_3001 = insertAtPosition_2511533001(head_3001, pos_3001, data2_3001);
93     printList_2511533001(head_3001);
94 }

```

Penjelasan:

Class InsertDLL_2511533001 berisi tiga fungsi utama untuk operasi penambahan node pada Double Linked List, yaitu insertBegin, insertEnd, dan insertAtPosition.

Pada insertBegin_2511533001, node baru langsung ditempatkan di posisi paling depan. Pointer next dari node baru diarahkan ke head yang lama, lalu kalau head lama tidak null, pointer prev head lama dibalik ke node baru. Node baru ini kemudian jadi head yang baru.

Untuk insertEnd_2511533001, cara kerjanya berbeda. Program menelusuri list dari head sampai ketemu node paling akhir, yaitu node yang next-nya sudah null. Setelah ketemu, node baru disambungkan di sana dengan menghubungkan next node terakhir ke node baru, dan prev node baru balik ke node terakhir tadi. Kalau listnya masih kosong, node baru langsung jadi head.

Pada insertAtPosition_2511533001, node baru disisipkan di posisi tertentu sesuai parameter yang dimasukkan. Kalau posisinya 1, prosesnya sama seperti insertBegin. Kalau bukan, program akan berjalan dulu menelusuri list sampai ke node tepat sebelum posisi yang dituju. Setelah ketemu, pointer prev dan next dari node baru diatur agar tersambung dengan benar ke

node sebelum dan sesudahnya. Jika posisi yang diminta melebihi panjang list, program akan mencetak pesan "Posisi tidak ada".

Terakhir, `printList_2511533001` digunakan untuk mencetak seluruh isi list dari head sampai akhir dengan format data `<->`. Di bagian main, dibuat DLL awal dengan isi 2, 3, dan 5, lalu dicoba ketiga operasi insert tersebut secara berurutan untuk memastikan semuanya berjalan sesuai yang diharapkan.

Output:

```
DLL Awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

2.3 PenelusuranDLL_2511533001

```
1 package pekan6_2511533001;
2
3 public class PenelusuranDLL_2511533001 {
4     // fungsi penelusuran maju
5     static void forwardTraversal(NodeDLL_2511533001 head_3001) {
6         // memulai penelusuran dari head
7         NodeDLL_2511533001 curr_3001 = head_3001;
8         // lanjutkan sampai akhir
9         while (curr_3001 != null) {
10            // print data
11            System.out.print(curr_3001.data_3001 + " <->");
12            // pindah ke node berikutnya
13            curr_3001 = curr_3001.next_3001;
14        }
15        // print spasi
16        System.out.println();
17        // fungsi penelusuran mundur
18        static void backwardTraversal(NodeDLL_2511533001 tail_3001) {
19            // mulai dari akhir
20            NodeDLL_2511533001 curr_3001 = tail_3001;
21            // lanjut sampai head
22            while (curr_3001 != null) {
23                // cetak data
24                System.out.print(curr_3001.data_3001 + " <->");
25                curr_3001 = curr_3001.prev_3001;
26            }
27            // cetak spasi
28            System.out.println();
29        }
30        public static void main(String[] args) {
31            // cetak DLL
32            NodeDLL_2511533001 head_3001 = new NodeDLL_2511533001(1);
33            NodeDLL_2511533001 second_3001 = new NodeDLL_2511533001(2);
34            NodeDLL_2511533001 third_3001 = new NodeDLL_2511533001(3);
35
36            head_3001.next_3001 = second_3001;
37            second_3001.prev_3001 = head_3001;
38            second_3001.next_3001 = third_3001;
39            third_3001.prev_3001 = second_3001;
40
41            System.out.println("Penelusuran maju:");
42            forwardTraversal (head_3001);
43
44            System.out.println("Penelusuran mundur:");
45            backwardTraversal (third_3001);
46        }
47    }
48 }
```

Penjelasan:

Class `PenelusuranDLL_2511533001` berisi dua fungsi penelusuran yang memanfaatkan keunggulan utama Double Linked List, yaitu bisa ditelusuri dari dua arah.

Fungsi `forwardTraversal` menelusuri list dari depan ke belakang. Dimulai dari node head, program terus berpindah ke node berikutnya lewat pointer `next_3001` sambil mencetak datanya, sampai tidak ada lagi node yang tersisa alias `curr_3001` sudah bernilai null.

Sebaliknya, backwardTraversal bekerja dari arah yang berlawanan. Parameter yang dimasukkan bukan head melainkan tail, yaitu node paling akhir. Dari sana program berjalan mundur menggunakan pointer prev_3001 sampai kembali ke awal list. Fungsi inilah yang tidak bisa dilakukan oleh Single Linked List karena tidak punya pointer prev.

Di bagian main, dibuat DLL sederhana dengan tiga node berisi nilai 1, 2, dan 3 yang disambungkan secara manual. Setelah list terbentuk, kedua fungsi penelusuran tadi dipanggil secara bergantian untuk membuktikan bahwa DLL memang bisa ditelusuri dari dua arah, maju maupun mundur.

Output:

```
Penelusuran maju:
1 <->2 <->3 <->
Penelusuran mundur:
3 <->2 <->1 <->
```

2.4 HapusDLL_2511533001

```
1 package pekanb_2511533001;
2
3 public class HapusDLL_2511533001 {
4     // fungsi menghapus node awal
5     public static NodeDLL_2511533001 delHead_2511533001(NodeDLL_2511533001 head_3001) {
6         if (head_3001 == null) {
7             return null;
8         }
9         NodeDLL_2511533001 temp_3001 = head_3001;
10        head_3001 = head_3001.next_3001;
11        if (head_3001 != null) {
12            head_3001.prev_3001 = null; }
13        return head_3001;
14    }
15    // fungsi menghapus di akhir
16    public static NodeDLL_2511533001 delLast_2511533001(NodeDLL_2511533001 head_3001) {
17        if (head_3001 == null) return null;
18
19        // jika hanya ada 1 node
20        if (head_3001.next_3001 == null) {
21            return null;
22        }
23
24        NodeDLL_2511533001 curr_3001 = head_3001;
25        while (curr_3001.next_3001 != null) {
26            curr_3001 = curr_3001.next_3001;
27        }
28
29        // Putus hubungan dengan node terakhir
30        if (curr_3001.prev_3001 != null) {
31            curr_3001.prev_3001.next_3001 = null;
32        }
33        return head_3001;
34    }
35    // fungsi menghapus node posisi tertentu
36    public static NodeDLL_2511533001 delPos_2511533001(NodeDLL_2511533001 head_3001, int pos_3001) {
37        // jika DLL kosong
38        if (head_3001 == null) {
39            return head_3001; }
40        NodeDLL_2511533001 curr_3001 = head_3001;
41        // telusuri sampai ke node yang akan dihapus
42        for (int i = 1; curr_3001 != null && i < pos_3001; ++i) {
43            curr_3001 = curr_3001.next_3001;
44        }
45        // jika posisi tidak ditemukan
46        if (curr_3001 == null) {
47            return head_3001; }
48        // update pointer
49        if (curr_3001.prev_3001 != null) {
50            curr_3001.prev_3001.next_3001 = curr_3001.next_3001; }
51        if (curr_3001.next_3001 != null) {
52            curr_3001.next_3001.prev_3001 = curr_3001.prev_3001; }
53        // jika yang dihapus head
54        if (head_3001 == curr_3001) {
55            head_3001 = curr_3001.next_3001;
56        }
57        return head_3001;
58    }
59    // fungsi mencetak DLL
60    public static void printList_2511533001(NodeDLL_2511533001 head_3001) {
61        NodeDLL_2511533001 curr_3001 = head_3001;
62        while (curr_3001 != null) {
63            System.out.print(curr_3001.data_3001 + " <->");
64            curr_3001 = curr_3001.next_3001;
65        }
66        System.out.println();
67    }
68    public static void main(String[] args) {
69        // buat sebuah DLL
70        NodeDLL_2511533001 head_3001 = new NodeDLL_2511533001(1);
71        head_3001.next_3001 = new NodeDLL_2511533001(2);
72        head_3001.next_3001.prev_3001 = head_3001;
73        head_3001.next_3001.next_3001 = new NodeDLL_2511533001(3);
```



```

71     head_3001.next_3001.next_3001.prev_3001 = head_3001.next_3001;
72     head_3001.next_3001.next_3001.next_3001 = new NodeDLL_2511533001(4);
73     head_3001.next_3001.next_3001.next_3001.prev_3001 = head_3001.next_3001.next_3001;
74     head_3001.next_3001.next_3001.next_3001.next_3001 = new NodeDLL_2511533001(5);
75     head_3001.next_3001.next_3001.next_3001.next_3001.prev_3001 = head_3001.next_3001.next_3001.next_3001;
76
77     System.out.print("DLL Awal:");
78     printList_2511533001(head_3001);
79
80     System.out.print("Setelah head dihapus: ");
81     head_3001 = delHead_2511533001(head_3001);
82     printList_2511533001(head_3001);
83
84     System.out.print("Setelah node terakhir dihapus: ");
85     head_3001 = delLast_2511533001(head_3001);
86     printList_2511533001(head_3001);
87
88     System.out.print("menghapus node ke 2: ");
89     head_3001 = delPos_2511533001(head_3001, 2);
90
91     printList_2511533001(head_3001);
92 }
93 }

```

Penjelasan:

Class HapusDLL_2511533001 berisi tiga fungsi penghapusan node pada Double Linked List, masing-masing untuk menghapus di awal, di akhir, dan di posisi tertentu.

Fungsi delHead_2511533001 menghapus node paling depan. Kalau list kosong langsung return null. Kalau tidak, head dipindah ke node berikutnya, lalu pointer prev dari head baru di-set null supaya tidak ada lagi referensi ke node yang sudah dihapus.

Fungsi delLast_2511533001 bekerja dengan menelusuri list sampai ketemu node paling akhir, yaitu yang next-nya sudah null. Setelah ketemu, pointer next dari node sebelumnya langsung diputus dengan di-set null. Kalau list hanya punya satu node, fungsi ini langsung return null karena listnya jadi kosong.

Fungsi delPos_2511533001 menghapus node di posisi tertentu sesuai parameter yang dimasukkan. Program menelusuri list sampai ke posisi yang dituju, lalu memutus koneksi node tersebut dari kedua sisinya — pointer next dari node sebelumnya diarahkan ke node sesudahnya, dan pointer prev dari node sesudahnya diarahkan balik ke node sebelumnya. Kalau posisi yang diminta tidak ada, list dikembalikan apa adanya tanpa perubahan. Kalau node yang dihapus ternyata adalah head, maka head diperbarui ke node berikutnya.

Di bagian main, dibuat DLL dengan isi 1 sampai 5, lalu ketiga fungsi hapus dicoba satu per satu. Hasilnya dicetak setiap kali operasi selesai untuk memastikan list berubah sesuai yang diharapkan.

Output:

```

DLL Awal:1 <->2 <->3 <->4 <->5 <->
Setelah head dihapus: 2 <->3 <->4 <->5 <->
Setelah node terakhir dihapus: 2 <->3 <->4 <->
menghapus node ke 2: 2 <->4 <->

```

2.5 Tugas

1. Lagu_2511533001

```
1 package pekan6_2511533001;
2
3 public class Lagu_2511533001 {
4     String judul_3001;
5     String penyanyi_3001;
6     Lagu_2511533001 next_3001;
7     Lagu_2511533001 prev_3001;
8
9     public Lagu_2511533001(String judul_3001, String penyanyi_3001) {
10         this.judul_3001 = judul_3001;
11         this.penyanyi_3001 = penyanyi_3001;
12         this.next_3001 = null;
13         this.prev_3001 = null;
14     }
15
16     public String getJudul_3001() { return judul_3001; }
17     public String getPenyanyi_3001() { return penyanyi_3001; }
18     public Lagu_2511533001 getNext_3001() { return next_3001; }
19     public Lagu_2511533001 getPrev_3001() { return prev_3001; }
20     public void setNext_3001(Lagu_2511533001 next_3001) { this.next_3001 = next_3001; }
21     public void setPrev_3001(Lagu_2511533001 prev_3001) { this.prev_3001 = prev_3001; }
22 }
```

Penjelasan:

Class `Lagu_2511533001` adalah representasi node dalam struktur Double Linked List. Setiap objek lagu menyimpan dua data yaitu `judul_3001` dan `penyanyi_3001`, serta dua pointer `next_3001` dan `prev_3001` yang masing-masing menunjuk ke lagu sesudah dan sebelumnya dalam playlist. Kedua pointer ini yang menjadikan struktur ini DLL, bukan Single Linked List.

Constructor menerima judul dan penyanyi sebagai parameter, lalu langsung menyimpannya ke atribut masing-masing. Pointer `next_3001` dan `prev_3001` diinisialisasi dengan `null` karena node yang baru dibuat belum terhubung ke node manapun.

Getter dan setter disediakan untuk setiap atribut supaya data bisa diakses dan dimodifikasi dari luar class secara terstruktur, tanpa harus mengakses atribut langsung.

2. Musik_2511533001

```
1 package pekan6_2511533001;
2 import java.util.*;
3
4 public class Musik_2511533001 {
5
6     Lagu_2511533001 head_3001 = null;
7     Lagu_2511533001 tail_3001 = null;
8
9     public void tambahLagu_3001(String judul_3001, String penyanyi_3001) {
10         Lagu_2511533001 baru_3001 = new Lagu_2511533001(judul_3001, penyanyi_3001);
11         if (head_3001 == null) {
12             head_3001 = baru_3001;
13             tail_3001 = baru_3001;
14         } else {
15             tail_3001.next_3001 = baru_3001;
16             baru_3001.prev_3001 = tail_3001;
17             tail_3001 = baru_3001;
18         }
19         System.out.println("Lagu berhasil ditambahkan!");
20     }
21
22     public void hapusLaguAwal_2511533001() {
23         if (head_3001 == null) {
24             System.out.println("Playlist kosong!");
25             return;
26         }
27         System.out.println("Lagu " + head_3001.getJudul_3001() + " berhasil dihapus");
28         head_3001 = head_3001.next_3001;
29         if (head_3001 != null) {
30             head_3001.prev_3001 = null;
31         } else {
32             tail_3001 = null;
33         }
34     }
35 }
```

```

36 public void tampilMaju_2511533001() {
37     if (head_3001 == null) {
38         System.out.println("Playlist kosong!");
39         return;
40     }
41     Lagu_2511533001 curr_3001 = head_3001;
42     System.out.println("=== Playlist Maju ===");
43     while (curr_3001 != null) {
44         System.out.println(curr_3001.getJudul_3001() + " - " + curr_3001.getPenyanyi_3001());
45         curr_3001 = curr_3001.next_3001;
46     }
47 }
48
49 public void tampilMundur_2511533001() {
50     if (tail_3001 == null) {
51         System.out.println("Playlist kosong!");
52         return;
53     }
54     Lagu_2511533001 curr_3001 = tail_3001;
55     System.out.println("=== Playlist Mundur ===");
56     while (curr_3001 != null) {
57         System.out.println(curr_3001.getJudul_3001() + " - " + curr_3001.getPenyanyi_3001());
58         curr_3001 = curr_3001.prev_3001;
59     }
60 }
61
62 public void cariLagu_2511533001(String judul_3001) {
63     if (head_3001 == null) {
64         System.out.println("Playlist kosong!");
65         return;
66     }
67     Lagu_2511533001 curr_3001 = head_3001;
68     boolean ketemu_3001 = false;
69     while (curr_3001 != null) {
70         if (curr_3001.getJudul_3001().equalsIgnoreCase(judul_3001)) {

```

```

71         System.out.println("Lagu ditemukan!");
72         System.out.println("Judul : " + curr_3001.getJudul_3001());
73         System.out.println("Penyanyi : " + curr_3001.getPenyanyi_3001());
74         ketemu_3001 = true;
75         break;
76     }
77     curr_3001 = curr_3001.next_3001;
78 }
79 if (!ketemu_3001) {
80     System.out.println("Lagu tidak ditemukan!");
81 }
82 }
83
84 public static void main(String[] args) {
85     Scanner input = new Scanner(System.in);
86     Musik_2511533001 musik_3001 = new Musik_2511533001();
87     int pilihan_3001;
88     do {
89         System.out.println("\n=== Playlist Musik NIM: 2511533001 ===");
90         System.out.println("1. Tambah Lagu");
91         System.out.println("2. Hapus Lagu Pertama");
92         System.out.println("3. Lihat Playlist (Maju)");
93         System.out.println("4. Lihat Playlist (Mundur)");
94         System.out.println("5. Cari Lagu");
95         System.out.println("6. Keluar");
96         System.out.print("Pilihan: ");
97         pilihan_3001 = input.nextInt();
98         input.nextLine();
99         switch (pilihan_3001) {
100             case 1:
101                 System.out.print("Judul Lagu: ");
102                 String judul_3001 = input.nextLine();
103                 System.out.print("Penyanyi: ");
104                 String penyanyi_3001 = input.nextLine();
105                 musik_3001.tambahLagu_3001(judul_3001, penyanyi_3001);
106                 break;
107             case 2:
108                 musik_3001.hapusLaguAwal_2511533001();
109                 break;
110             case 3:
111                 musik_3001.tampilMaju_2511533001();
112                 break;
113             case 4:
114                 musik_3001.tampilMundur_2511533001();
115                 break;
116             case 5:
117                 System.out.print("Masukkan judul lagu: ");
118                 String cari_3001 = input.nextLine();
119                 musik_3001.cariLagu_2511533001(cari_3001);
120                 break;
121             case 6:
122                 System.out.println("Program selesai");
123                 break;
124             default:
125                 System.out.println("Pilihan tidak valid!");
126         }
127     } while (pilihan_3001 != 6);
128     input.close();
129 }
130 }

```

Penjelasan:

Class Musik_2511533001 adalah class yang mengelola playlist lagu menggunakan struktur DLL. Berbeda dengan Lagu_2511533001 yang hanya berperan sebagai node, class ini yang mengatur bagaimana node-node tersebut dihubungkan dan dimanipulasi. Ada dua atribut utama di sini yaitu head_3001 sebagai penanda lagu paling depan dan tail_3001 sebagai penanda lagu paling belakang, keduanya awalnya bernilai null karena playlist masih kosong.

Fungsi tambahLagu_2511533001 selalu menyisipkan lagu baru di posisi paling akhir playlist. Kalau playlist masih kosong, lagu pertama langsung jadi head sekaligus tail. Kalau sudah ada isinya, pointer next dari tail lama disambungkan ke lagu baru, lalu pointer prev lagu baru diarahkan balik ke tail lama, kemudian tail diperbarui ke lagu baru tadi.

Fungsi hapusLaguAwal_2511533001 menghapus lagu di posisi paling depan dengan cara memindahkan head ke node berikutnya, lalu memutus pointer prev dari head baru supaya tidak ada lagi referensi ke node yang sudah dihapus. Kalau setelah penghapusan playlist jadi kosong, tail juga ikut di-set null.

tampilMaju_2511533001 dan tampilMundur_2511533001 adalah dua fungsi yang paling mencerminkan keunggulan DLL. tampilMaju_2511533001 berjalan dari head ke tail lewat pointer next, sedangkan tampilMundur_3001 berjalan sebaliknya dari tail ke head lewat pointer prev. Fungsi mundur ini yang tidak bisa dilakukan kalau hanya pakai Single Linked List.

Fungsi cariLagu_2511533001 menelusuri playlist dari head sambil membandingkan judul tiap node dengan kata kunci yang dimasukkan menggunakan equalsIgnoreCase, jadi pencarian tidak terpengaruh huruf kapital atau tidak. Kalau ketemu langsung ditampilkan dan loop berhenti, kalau sampai akhir tidak ada yang cocok maka program mencetak pesan tidak ditemukan.

Bagian main menjalankan program dengan loop do-while yang terus menampilkan menu sampai user memilih opsi 6 untuk keluar. Setiap pilihan memanggil fungsi yang sesuai dari objek musik_3001.

Output:

```
=== Playlist Musik NIM: 2511533001 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu: Evaluasi
Penyanyi: Hindia
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511533001 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu: Secukupnya
Penyanyi: Hindia
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511533001 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu: Rumah ke Rumah
Penyanyi: Hindia
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511533001 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
=== Playlist Maju ===
Evaluasi - Hindia
Secukupnya - Hindia
Rumah ke Rumah - Hindia
```

=== Playlist Musik NIM: 2511533001 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 4

=== Playlist Mundur ===

Rumah ke Rumah - Hindia

Secukupnya - Hindia

Evaluasi - Hindia

=== Playlist Musik NIM: 2511533001 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 5

Masukkan judul lagu: Secukupnya

Lagu ditemukan!

Judul : Secukupnya

Penyanyi : Hindia

=== Playlist Musik NIM: 2511533001 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 2

Lagu Evaluasi berhasil dihapus

=== Playlist Musik NIM: 2511533001 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 3

=== Playlist Maju ===

Secukupnya - Hindia

Rumah ke Rumah - Hindia

=== Playlist Musik NIM: 2511533001 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 6

Program selesai

BAB III

PENUTUP

3.1 Kesimpulan

Dari praktikum mengenai Double Linked List, dapat disimpulkan bahwa DLL merupakan struktur data linier yang setiap nodenya memiliki dua pointer sekaligus, yaitu next yang menunjuk ke node berikutnya dan prev yang menunjuk ke node sebelumnya. Berbeda dengan Single Linked List yang hanya bisa ditelusuri satu arah, DLL memungkinkan traversal dilakukan dari dua arah, baik maju maupun mundur.

Operasi dasar seperti insert, delete, dan traverse pada DLL dapat dilakukan di berbagai posisi, yaitu di awal, di akhir, maupun di posisi tertentu. Adanya pointer prev membuat operasi penghapusan dan penyisipan di posisi tertentu menjadi lebih efisien karena tidak perlu menelusuri ulang dari awal untuk menemukan node sebelumnya. Penerapan DLL pada praktikum ini, seperti pada studi kasus playlist musik, menunjukkan bahwa konsep ini relevan digunakan dalam situasi yang membutuhkan akses data dua arah secara fleksibel.

Dengan demikian, Double Linked List tidak hanya penting dipahami secara teori, tetapi juga memiliki manfaat nyata dalam pengembangan sistem yang memerlukan pengelolaan data secara dinamis dan efisien.

3.2 Saran

Sebaiknya dosen menyelenggarakan sesi pra-praktikum di kelas sebagai pengantar materi, agar mahasiswa memperoleh pemahaman awal yang lebih baik. Dengan demikian, potensi kepanikan atau kesalahan selama praktikum dapat diminimalkan. Disarankan agar materi praktikum dibagikan terlebih dahulu melalui platform iLearn, sehingga mahasiswa memiliki waktu yang cukup untuk mempelajari dan mempersiapkan diri sebelum pelaksanaan praktikum.

DAFTAR PUSTAKA

[1] Oracle, “LinkedList Class (Java Platform SE Documentation),” [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/java/util/LinkedList.html>